Event Loop in JavaScript

👨‍💼 **Interviewer:**

"Suppose I give you `setTimeout`, `Promise`, and a normal `console.log` — can you tell me the exact order in which they'll execute and why? Explain using the Event Loop."

```javascript
console.log('Start');

setTimeout(() => {
  console.log('Timeout');
}, 0);

Promise.resolve().then(() => {
  console.log('Promise');
});

console.log('End');
```

👩‍💻 **Candidate (Using What / Why / How):**

🔹 **What is the Event Loop?**

Event Loop is a **mechanism in JavaScript runtime** (like in browsers or Node.js) that manages **execution of sync and async code**, allowing JS (which is single-threaded) to handle non-blocking operations like timers, DOM events, or API calls.

🔹 **Why do we need an Event Loop?**
Because JavaScript can't run multiple threads simultaneously (like C++ or Java), so the **event loop allows it to handle concurrency** using a queue system and **callback mechanisms**.

🔹 **How does it work?**
The Event Loop keeps checking:

1. **Call Stack** – for synchronous code.
2. **Microtask Queue** – for Promises, `queueMicrotask`.
3. **Macrotask Queue** – for `setTimeout`, `setInterval`, etc.

The loop runs like:

- Finish all sync code on the call stack.
- Then run **all microtasks**.
- Then pick **one macrotask** (e.g., from `setTimeout`) and repeat.

Output:

```
Start
End
Promise
Timeout
```

mycodeshala

❗ **Explanation:**

- `console.log('Start')` → **Sync → goes first**
- `setTimeout` → **Macrotask → scheduled**
- `Promise.then()` → **Microtask → after sync**
- `console.log('End')` → **Sync → goes next**
- **Then → Microtask (`Promise`)**
- **Then → Macrotask (`setTimeout`)**

📟 **Real-world Analogy**

Think of:

- 🧾 **Call Stack = Chef handling orders one-by-one.**
- 📮 **Microtasks = Special urgent requests (like manager's orders).**
- 🕰️ **Macrotasks = Regular customer orders queued by time.**

mycodeshala

🔟 Follow-Up Questions on Event Loop

1. What's the difference between the **call stack** and the **event loop**?
2. What's the difference between **microtasks** and **macrotasks**?
3. Predict the output:

```javascript
Promise.resolve().then(() => {
  console.log('A');
  setTimeout(() => console.log('B'), 0);
});
setTimeout(() => console.log('C'), 0);
console.log('D');
```

4. What is the role of the job queue (microtask queue)?

5. How does queueMicrotask() behave compared to .then()?

6. Is async/await part of microtask queue?

7. Explain the difference between process.nextTick() and Promise.then() in Node.js.

8. Can you block the event loop? Give an example.

9. How does the event loop differ between browser and Node.js?

10. Why does setting setTimeout(fn, 0) not always run immediately?

mycodesha